

Build a Modern Trading Journal Web Application

Create a modern, responsive web application for **personal trade journaling, review, and performance analysis**.

The application is intended for an active discretionary trader trading multiple strategies across **TradFi and crypto**, initially including:

- CFDs such as Gold/XAUUSD
- Nasdaq CFDs
- Dow Jones CFDs
- Other index/commodity CFDs
- Crypto spot
- Crypto perpetuals/futures

This is an **MVP**, not a finished commercial SaaS product.

The priority is to create a polished, genuinely usable foundation that can later be expanded with advanced analytics, broker imports, backtesting, market-data enrichment, and AI-assisted trading analysis.

Product principles

1. Make entering and reviewing trades fast.
2. Store trading data in a structured form rather than relying primarily on free-text notes.
3. Support multiple trading strategies. Do NOT design the application specifically around ORB or any single strategy.
4. Support trades based on setups and multiple confluences.
5. Model trades using individual executions/fills so scaling in and scaling out work correctly.
6. Financial and trading calculations must be deterministic and testable.
7. Keep the architecture simple.
8. Do not build speculative functionality that is not required for the MVP.
9. The UI should feel like a premium modern trading application, not an accounting application or generic admin dashboard.
10. Optimize for desktop use first, while remaining responsive.

Core domain model

Use an execution-based trading model.

A Trade should conceptually contain:

- Account
- Instrument
- Asset class

- Direction
- One or more executions
- Trade plan/risk information
- Strategy
- Setup
- Zero or more confluences
- Tags
- Notes
- Screenshots/attachments
- Trade review

Do NOT make a single entry price and exit price the authoritative source of a trade.

Instead, individual executions are the authoritative transaction data and average entry, average exit, position size, realized P&L, etc. should be derived from them.

Execution

Every execution/fill should support at least:

- Date and time
- Buy or sell
- Price
- Quantity
- Quantity unit
- Commission
- Fees

Examples of quantity units include:

- Lots
- Contracts
- Units
- BTC/ETH/etc.

The data model must allow multiple entries and multiple exits.

Example:

BUY 0.50 lot XAUUSD @ 3345.20
 BUY 0.50 lot XAUUSD @ 3342.80
 SELL 0.40 lot XAUUSD @ 3351.10
 SELL 0.60 lot XAUUSD @ 3357.40

The application should derive the relevant average entry/exit and performance values.

Use appropriate decimal arithmetic for prices, quantities and monetary calculations. Do not rely on binary floating-point arithmetic for authoritative financial calculations.

Date and time

Date/time handling is essential.

Every execution must have an exact timestamp.

Store timestamps consistently using an appropriate UTC-based representation and display them according to the relevant/user-selected timezone.

The architecture must not assume that a fixed UTC time always represents a particular trading session because daylight-saving-time changes must be supportable later.

Derive useful values such as:

- Trade opened at
- Trade closed at
- Trade duration
- Day of week
- Time of day

Do not yet build complicated session analytics.

Accounts

Allow creation and management of trading accounts.

An account should support fields such as:

- Account name
- Broker/exchange
- Account type
- Base currency
- Optional starting balance
- Active/inactive status

Examples might be:

- CFD Live Account
- Prop Account
- Binance Crypto
- Bybit Crypto

Do not build broker API synchronization yet.

Instruments

Support a flexible instrument model.

Initial examples:

- XAUUSD CFD
- NAS100 CFD
- US30 CFD
- BTCUSDT perpetual
- ETHUSDT perpetual

Do not assume that "Nasdaq" is itself an executable instrument. Keep the traded instrument distinct from its underlying market where appropriate.

The model should be extensible to futures, forex, stocks and other instruments later without requiring a fundamental redesign.

Trade planning and risk

Support planned trade information including:

- Planned entry
- Initial stop loss
- Profit target
- Planned position size
- Planned monetary risk
- Planned account risk percentage
- Optional planned reward/risk ratio

Keep planned values separate from actual execution values.

This distinction is important.

For example:

Planned entry: 3345.00

Actual average entry: 3346.20

Planned risk: \$400

Actual risk: \$445

Strategies and setups

The journal must support multiple strategies.

Examples could include:

- ORB
- Large Level Reversal
- Breakout
- Trend Continuation

Do NOT hard-code these as the only strategies.

Users must be able to create/edit their own strategies and setups.

A strategy may contain multiple setups.

Confluences

Confluences are important and should be structured rather than stored only as free text.

Examples:

- Daily support/resistance
- Weekly support/resistance
- Previous day high/low
- VWAP
- Moving average
- Fibonacci level
- Liquidity sweep
- Market structure
- Volume
- Momentum indicator
- Higher-timeframe trend

Allow users to define their own confluences.

A trade can contain zero, one, or many confluences.

Design this so future analytics can answer questions such as:

"How do my Gold trades perform when VWAP and a weekly level are both present?"

Do not build that advanced analytics engine yet; preserve the data required to support it later.

Trade review

Provide a post-trade review section.

Support at least:

- Review notes
- What went well
- What went wrong
- Mistakes
- Rule adherence
- Optional trade quality/rating
- Lessons learned

Keep this relatively lightweight for the MVP.

Screenshots

Allow screenshots/images to be attached to a trade.

The trade detail page should prominently display attached chart screenshots.

Design the storage abstraction cleanly so object/cloud storage can be used.

Journal

Create a main Journal page showing recorded trades.

It should be optimized for quickly scanning trading history.

Useful columns include:

- Date
- Instrument
- Direction
- Strategy/setup
- Average entry
- Average exit
- Position size
- P&L
- R
- Duration
- Account

Provide basic filtering for:

- Date range
- Account
- Instrument
- Asset class
- Long/short
- Strategy
- Setup
- Confluences/tags
- Winning/losing trades

Do not build an overly complicated query builder for the MVP.

Dashboard

Create a visually polished but intentionally simple dashboard.

Include useful headline statistics such as:

- Net P&L
- Number of trades
- Win rate
- Average winning trade
- Average losing trade
- Average R
- Profit factor

Include a small number of useful visualizations, for example:

- Equity/P&L curve
- P&L by day
- Performance by strategy
- Recent trades

Do not fill the dashboard with dozens of charts merely to make it look sophisticated.

Every statistic must be calculated from actual journal data.

Trade detail page

Create a strong trade-detail experience.

A possible structure is:

Header

Instrument · Direction · Result in R · P&L

Overview

Account
Opened
Closed
Duration
Average entry
Average exit
Position size
Gross P&L
Net P&L
Initial risk
R result

Executions

Show all individual fills in chronological order.

Plan

Entry idea
Stop
Targets
Risk

Strategy

Strategy
Setup
Confluences
Tags

Charts/screenshots

Display uploaded chart images.

Review

Notes
Mistakes
Rule adherence
Lessons

Use tabs or another clean information architecture if this becomes too dense.

Add/Edit Trade experience

This is one of the most important parts of the application.

It must be fast and pleasant to use.

Organize the form logically rather than presenting one giant form containing every database field.

A possible flow:

1. Account and instrument
2. Executions
3. Risk/plan
4. Strategy/setup/confluences
5. Notes/screenshots
6. Save

Make simple trades simple.

A trade with one entry and one exit should not feel complicated merely because the underlying model supports multiple executions.

Calculated values

Where sufficient information exists, derive values rather than requiring the user to manually enter them.

Examples:

- Average entry
- Average exit
- Total quantity
- Gross P&L
- Net P&L
- Duration
- R-multiple
- Win/loss/breakeven

Clearly separate:

- User-entered data
- Imported data
- Derived/calculated data

Do not allow users to accidentally create conflicting authoritative values.

UI/UX direction

Create a premium, modern trading-oriented UI.

Characteristics:

- Clean
- Data-rich without feeling crowded
- Strong typography
- Clear visual hierarchy
- Modern cards/tables/forms
- Excellent spacing
- High readability
- Professional rather than flashy
- Desktop-first responsive design
- Dark mode should look particularly good
- Light mode can also be supported if practical

Take conceptual inspiration from modern trading terminals, analytics dashboards, and premium trading journals, but do not clone another product's design.

Use color purposefully for concepts such as:

- Profit/loss
- Long/short
- Risk
- Status

Do not overuse gradients, neon effects, excessive animations, glassmorphism, or decorative elements.

Navigation

Start with a simple application structure:

- Dashboard
- Journal
- Add Trade
- Strategies
- Accounts
- Settings

Add other pages only when required.

Technical direction

Use a modern, maintainable web stack appropriate for this application.

Prefer:

- TypeScript
- React/Next.js or equivalent modern framework
- PostgreSQL or equivalent relational database
- A mature ORM
- A high-quality component system
- Appropriate charting libraries
- Managed authentication
- Object storage for screenshots

Keep the architecture simple.

Do NOT introduce:

- Microservices
- Kubernetes
- Message queues
- Event-driven infrastructure
- Vector databases
- Separate AI infrastructure
- Separate analytics services

unless they become demonstrably necessary later.

Testing and correctness

Trading calculations require automated tests.

At minimum, test:

- Single entry/single exit
- Multiple entries
- Multiple exits
- Scaling in
- Scaling out
- Long trades
- Short trades
- Commissions and fees
- Average entry
- Average exit
- Gross P&L

- Net P&L
- R calculation
- Trade duration

Never invent financial calculation rules when requirements are ambiguous. Keep the implementation explicit and flag important ambiguities.

Explicitly outside the MVP

Do NOT implement these now:

- Broker API synchronization
- Automated broker imports
- Advanced market-data integration
- Advanced MFE/MAE analysis
- Trade replay
- Backtesting engine
- AI trading coach
- AI agents
- Natural-language analytics
- Automated strategy analysis
- Prop-firm evaluation tooling
- Social/community functionality
- Billing/subscriptions
- Native mobile applications

The architecture may remain compatible with these future capabilities, but do not build infrastructure for them prematurely.

Development philosophy

This project should use a pragmatic agile/vibe-coding approach.

Prefer:

working software → use it → identify problem → small improvement → test → repeat

over trying to design the final application upfront.

When choosing between:

A. a simple implementation that satisfies the current requirement

and

B. a sophisticated generalized system intended to support hypothetical future requirements

prefer **A**, unless doing so would compromise the integrity of the core trading data.

Be particularly conservative around:

- Database/data model
- Monetary calculations
- Position calculations
- Timestamp/timezone handling
- Authentication/security
- Database migrations

Be highly iterative around:

- UI
- Styling
- Dashboard layout
- Forms
- Navigation
- Charts
- UX conveniences

In short:

Vibe-code what is cheap to change. Engineer carefully what could corrupt data or produce incorrect trading conclusions.

First implementation milestone

Do not attempt to implement the entire vision in one pass.

First build a coherent vertical slice:

1. Application shell
2. Navigation
3. Basic authentication if appropriate
4. Database
5. Account management
6. Instrument management
7. Create a trade
8. Add entry and exit executions
9. Save the trade
10. Calculate basic trade values
11. Display it in the journal
12. Open a trade-detail page

13. Edit the trade

Seed the development environment with several realistic example trades, including:

- XAUUSD CFD trade with one entry/exit
- NAS100 CFD trade with multiple entries and partial exits
- BTCUSDT perpetual trade

Use realistic-looking but fictional data.

Once this vertical slice works well, stop.

Do not automatically proceed to every feature described above.

Present the working implementation and identify:

- What was implemented
- Important assumptions made
- Anything ambiguous that needs a product decision
- Tests performed
- Suggested next 3 improvements

The objective is to get a **small, polished, working trading journal into the user's hands quickly**, then evolve it based on actual usage.